

R Code from Chapter 3

The Four Functions

As prefaced in code from Chapter 1, when you are looking at solving probabilities for common distributions, base R (and packages) have a family of functions used for particular items. The four functions are as follows:

- `d_____()`: used for point probabilities (i.e. $P(X = x)$)
- `p_____()`: used for cumulative probabilities (i.e. $P(X \leq x)$)
- `q_____()`: used for identifying quantiles (i.e. x for which $P(X \leq x) = q$)
- `r_____()`: used to generate observations from distribution (i.e. $X \sim \text{_____}()$)

The abbreviation for the distribution you are looking at goes inside the blanks. For discrete RVs, here are those abbreviations:

- `binom`: binomial distribution with parameters $size = n$ and $prob = p$
- `geom`: geometric distribution with parameters $prob = p$
- `hyper`: hypergeometric distribution with parameters m , n , and k

We will talk about the continuous RVs later, but they follow a similar format. Let's explore how to use these functions when we are working with $X \sim \text{Bin}(n = 50, p = 0.3)$. When using the functions, we want to specify the n and p parameters in the function as secondary arguments.

Suppose we cared about the probability of 4 successes for this random variable X . Then, we can use the following R code to find this point probability:

```
dbinom(x = 4, size = 50, prob = 0.3)
```

```
[1] 0.0001397297
```

Verify that this answer is correct by doing the probability by-hand – you should get the same result!

Now let's say we want to find a cumulative probability, such as the probability of having no more than 2 successes. Then, we can use the `pbinom` function as follows:

```
pbinom(q = 2, size = 50, prob = 0.3)
```

```
[1] 4.449916e-06
```

Note that $x = q$ when going from the `dbinom()` to `pbinom()` function. Say we want to find the probability of having greater than 2 successes. Then, we can find this one of two ways:

```
# Complement rule
1 - pbinom(q = 2, size = 50, prob = 0.3)
```

```
[1] 0.9999956
```

```
# Use of lower.tail = F
pbinom(q = 2, size = 50, prob = 0.3, lower.tail = F)
```

```
[1] 0.9999956
```

Functionally, they give you the same result. The `lower.tail = F` argument tells the function that you want to look at the upper tail probability instead of the lower tail probability.

Lastly, if I want to find the probability between 2 and 4 successes, then I can just write code that finds both $P(X \leq 4)$ and $P(X \leq 2)$, and then takes a difference of these two. Hence, we get:

```
pbinom(q = 4, size = 50, prob = 0.3) - pbinom(q = 1, size = 50, prob = 0.3)
```

```
[1] 0.000171524
```

If we have an inclusive lower bound, we will need to lower the x by 1. This is because we are removing the probability of having some value of x for values below 2 in order to get our probability. If we have $q = 2$, we will only get the probability of $x = 3, 4$.

Note: Watch out for whether values are included or excluded in the inequalities. If excluded, you will need to adjust your value of x .

- Upper bound is exclusive: reduce x by 1
- Lower bound is exclusive: keep x at a

We may also be curious about the value of x in which the percentage of the distribution beneath that value is a proportion of p . Here, we can use the `q_()` to find this value of x given some percentile p (i.e. $P(X \leq x) = p$). For example, suppose we wanted to find the number of successes for which 50% of the distribution is beneath it. Then, we can use the following code:

```
qbinom(p = 0.5, size = 50, prob = 0.3)
```

```
[1] 15
```

The x value it returns is the first value of x for which $P(X \leq x) \geq p$. This can be verified if we run:

```
pbinom(q = 14, size = 50, prob = 0.3)
```

```
[1] 0.4468316
```

```
pbinom(q = 15, size = 50, prob = 0.3)
```

```
[1] 0.5691784
```

Lastly, if we want to generate/simulate n observations from a distribution, we can use the `r_()` function to do so. As such, if we want to generate 100 observations from the $Bin(n = 50, p = 0.3)$ distribution, we can write this code:

```
rbinom(n = 100, size = 50, p = 0.3)
```

```
[1] 14 16 17 13 9 13 19 18 8 10 14 15 11 10 13 13 16 18 15 13 18 14 19 16 18
[26] 6 14 16 17 14 17 15 15 15 18 16 11 15 21 12 12 21 14 12 16 15 10 15 9 20
[51] 15 18 15 13 12 14 19 16 12 14 13 17 21 10 17 19 10 11 17 24 12 17 13 15 18
[76] 18 12 21 12 17 17 15 15 13 16 12 18 14 14 11 12 12 16 15 14 15 22 18 9 13
```

Note that the n argument that R uses here is different from the n parameter used universally in all probability problems involving a binomial distribution. Nonetheless, if we run it, we get 100 observations that come from the specified binomial distribution.

Generating and simulating n observations can also come from any discrete RV with finite values for the support – we just need the values of the support, their probabilities, and the `sample()` function. Suppose we evaluate the number of days someone runs in a week, with $x \in \{0, 1, \dots, 7\}$ and some $P(X = x)$ in which the sum of probabilities for values in the support add up to 1. Hence, we can do the following:

1. Define values of support using a vector we assign to x
2. Define probabilities for each value of support using a vector we assign to p
3. Use `sample()` function on x and p , with a size indicating the number of observations we want and replace being set to true.

```
x = c(0, 1, 2, 3, 4, 5, 6, 7)
p = c(0.0, 0.01, 0.09, 0.2, 0.25, 0.3, 0.13, 0.02)

sample(x, size = 100, replace = T, prob = p)
```

```
[1] 4 2 2 5 5 2 5 4 4 4 6 5 5 3 4 5 3 3 5 3 6 5 4 5 4 3 3 1 3 2 6 2 5 5 5 3 6
[38] 4 6 5 3 5 5 6 3 5 5 7 5 5 4 5 5 5 5 5 5 3 6 5 5 4 2 6 5 2 5 3 2 5 5 3 4 6
[75] 5 4 4 3 4 4 4 2 5 3 3 4 3 4 5 3 5 6 4 4 4 5 7 5 5 3
```

Again, we get observations from this specific random variable.